

Numerical Analysis

Lab 1

Giuseppe Calì

A.Y. 2024-2025

September 23, 2024

Table of contents

1 Introduction

2 The lab tool: MATLAB

3 Basic MATLAB usage

- Basic commands
- First MATLAB/Octave - Exercises

4 Floating-point numbers

- Representation of floating-point numbers
- Floating-point arithmetic

- info about me:
 - PhD student in Mathematical Models and Methods (Mathematics Department)
 - Research topics: Advanced Modeling of Ionic Propulsion for Atmospheric Flight
 - email: giuseppe.calio@polimi.it
- 10 labs = 20 h
- material on webeep.polimi.it: lecture slides and printable A4 version(.pdf), lecture excercises (.m), homework excercises(.m);
- approximation of function zeros, functions and integrals, numerical resolution of linear systems and ODEs

Table of contents

1 Introduction

2 The lab tool: MATLAB

3 Basic MATLAB usage

- Basic commands
- First MATLAB/Octave - Exercises

4 Floating-point numbers

- Representation of floating-point numbers
- Floating-point arithmetic

The lab tool: MATLAB I

We want to focus on the mathematics with low programming effort.

MATLAB(MATrix LABoratory) is an interactive software for:

- scientific computing
- statistical analysis
- vector and matrix computations
- symbolic computation
- graphics

Some MATLAB characteristics:

- graphical user interface;
- no need of compiling;
- code modularity: functions and scripts;
- simple graphics commands;
- I/O: loading and saving files, variables, and data.

Open source alternative: **Octave** - www.octave.org

Introduction to MATLAB:

- GUI description
- Basic MATLAB usage
- Vector and matrices
- Vector operations
- Matrix operations
- Plotting
- Function handles
- Logicals
- Control flow statements
- M-files

Table of contents

1 Introduction

2 The lab tool: MATLAB

3 Basic MATLAB usage

- Basic commands
- First MATLAB/Octave - Exercises

4 Floating-point numbers

- Representation of floating-point numbers
- Floating-point arithmetic

`ver` shows the Matlab version
we are working on

```
>> ver
-----
MATLAB Version: 23.2.0.2485118 (R2023b) Update 6
MATLAB License Number: 924786
```

`help` shows how to use a
function

```
>> help sqrt
sqrt Square root.
sqrt(X) is the square root of the elements of X. Complex
results are produced if X is not positive.

See also sqrtm, realsqrt, hypot.
```

`doc` displays online
documentation about a
command

```
>> doc sqrt
>>
```


Default Variables and Mathematical Functions

Some constants are allocated
by default

```
octave:1> pi  
ans = 3.1416  
octave:2> i  
ans = 0 + 1i  
octave:3> inf  
ans = Inf
```

Default Mathematical Functions

Matlab provides a large number of standard elementary mathematical functions. To have more information, type the command: `help elfun`

Number format

It is convenient to change the default format used by MATLAB to represent the number. Variables visualization can be changed by using the command `format` as following:

- **format short**

fixed comma with 5 digits(default): `pi=3.1416`

- **format long**

fixed comma with 15 digits(default): `pi=3.14159265358979`

- **format short e**

mobile comma with 5 digits(default): `pi=3.1416e+00`

- **format long e**

mobile comma with 15 digits(default): `pi=3.141592653589793e+00`

Exercise 1.1

Define the row vector

$$\underline{v}_k = [1, 9, 25, \dots, (2k+1)^2] \in \mathbb{R}^{k+1}$$

with $k = 8$ using the following strategies:

- 1 A for loop to define one by one each element of the vector.
- 2 The vector syntax to build it in just one shot.

Exercise 1.2

Define a function which, for an input value k , returns the corresponding vector $\underline{v}_k$ as defined in the previous exercise.

Exercise 1.3

Using the function of the previous exercise write another function that returns, for a generic value k , the $2(k+1) \times 2(k+1)$ matrix

$$M_k = \begin{bmatrix} 2 & 0 & 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & \sqrt[2]{2} & 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & \sqrt[3]{2} & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & \sqrt[4]{2} & 0 & 0 & \dots & 0 & 9 \\ 0 & 0 & 0 & 0 & \sqrt[5]{2} & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \sqrt[6]{2} & \dots & 0 & 25 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \dots & \sqrt[2k+1]{2} & 0 \\ 1 & 1 & 9 & 9 & 25 & 25 & \dots & (2k+1)^2 & (2k+1)^2 \end{bmatrix}.$$

Table of contents

1 Introduction

2 The lab tool: MATLAB

3 Basic MATLAB usage

- Basic commands
- First MATLAB/Octave - Exercises

4 Floating-point numbers

- Representation of floating-point numbers
- Floating-point arithmetic

Representation of floating-point numbers I

Exercise 1.4

Compare the results of the following code segments:

Code A

```
x = 0;
while (x ~= 1)
    x = x + 1/16
end
```

Code B

```
x = 0;
while (x ~= 1)
    x = x + 0.1
end
```

Solution Exercise 1.4.

Code A works as expected: the **while** loop is repeated 16 times, and the final value of **x** is 1.

Instead, code B does **not** work as expected, and results in an infinite loop.

Representation of floating-point numbers III

Computing environments use floating-point arithmetic, which involves a **finite set of numbers** with **finite precision**.

Given a real number x , the typical floating-point implementation represents its identifier $fl(x)$ in the following **normalized** form:

$$fl(x) = (-1)^s \cdot (0.a_1 a_2 \dots a_t) \cdot b^e$$

where

- s is the **sign**;
- b is the **base** of the system;
- e is the **exponent**, $e \in [L, U]$, $L < 0$, $U > 0$;
- $m = a_1 a_2 \dots a_t$, with $0 \leq a_i \leq b - 1$, is the **mantissa** or **fractional part**. To have a **unique representation** for each number we force $a_1 \neq 0$.

Representation of floating-point numbers IV

MATLAB represent floating point numbers in **double-precision** format:

- binary representation: $b = 2$;
- 64-bit words, with
 - 1 bit for the sign: 0 (+) or 1 (-);
 - $t = 52$ bits for the mantissa: $0 \leq m < 2^{52} \rightarrow$ limitation on the **precision** of the arithmetic;
 - 11 bits for the exponent: $L = -1021, U = 1023 \rightarrow$ limitation on the **range** of the arithmetic;

Representation of floating-point numbers V

- the minimum and the maximum normalized machine numbers (in absolute value) that can be represented in double precision can be obtained in MATLAB with

$$\mathbf{realmin} \quad \% = 2^{1-L} = 2.2251 \cdot 10^{-308}$$

$$\mathbf{realmax} \quad \% = (1 - 2^{-t}) \cdot 2^U = 1.7977 \cdot 10^{308}.$$

- Any computation trying to produce a value larger than `realmax` is said to **overflow**. The result is set to `Inf`.
- Any computation trying to produce a value smaller than the minimum number is said to **underflow**. If possible the result is treated in a special way (denormalized number), otherwise is set to 0.
- For any computation trying to produce a value that is undefined the result is called NaN. (e.g. `0/0`, `Inf/Inf`, `Inf-Inf`).

Representation of floating-point numbers VI

The different behaviour of Codes A and B can be now explained: the binary representation of $1/16$ is

$$\frac{1}{16} = 2^{-4}$$

and is therefore stored exactly in a double precision representation. Instead, the binary representation of $1/10$ is

$$\frac{1}{10} = 2^{-4} + 2^{-5} + 2^{-8} + 2^{-9} + 2^{-12} + 2^{-13} + \dots$$

and **cannot be stored exactly** in any finite-precision representation. $\square$

Floating-point arithmetic I

Machin epsilon

Definition 1.1 (Machine epsilon)

The smallest positive machine number that, if added to 1, gives a result greater than 1.

$$\text{eps} = 2^{-t}$$

Note that `eps` is different from `realmin`.

In MATLAB you can get the machine number by using the `eps` command.

Definition 1.2

Roundoff Error The **roundoff error**, namely the error due to the finite precision of the arithmetic, is bounded by the **machine epsilon**:

$$\frac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \text{eps} \quad \text{with } |x| \neq 0$$

Exercise 1.5 (Machine epsilon)

Find the machine epsilon by implementing an **ad hoc procedure**.
Comment and justify the obtained results.

Floating-point arithmetic

Overflow and associative property

Exercise 1.6 (Overflow and associative property)

```
realmax
a = 1.0e+308;
b = -a;
c = 1.1e+308;
(a + b) + c
(a + c) + b
```

Solution Exercise 1.6. While commutative property still holds ($a + b = b + a$ and $a \cdot b = b \cdot a$), associative property is no longer valid within floating-point arithmetic ($(a + b) + c \neq a + (b + c)$). The **order** in which the operations are performed sometimes matters, especially when overflow or underflow exception may occur! $\square$

Exercise 1.7 (Numerical cancellation)

Consider the following function

$$f(x) = \frac{e^x - 1}{x}$$

- a Evaluate $f(x)$ for values of x around zero (try with $x_k = 10^{-k}$, $k \in [1, 20]$). What do you obtain? Explain the results.
- b Propose an approach for fixing the problem. (Hint: Use Taylor expansions to get an approximation of $f(x)$ around $x = 0$.)
- c How many terms in the Taylor expansion are needed to get double precision accuracy (16 decimal digits) $\forall x \in [0, 1/2]$?

Table of contents

5 Homework

Homework: Floating-point arithmetic

Error propagation

Homework 1.1 (Error propagation)

The sequence

$$1, \frac{1}{3}, \frac{1}{9}, \dots, \frac{1}{3^n}, \dots$$

can be generated with the following recursive relations

$$\begin{cases} p_n = \frac{10}{3}p_{n-1} - p_{n-2} \\ p_1 = \frac{1}{3}, p_0 = 1 \end{cases} \quad \begin{cases} q_n = \frac{1}{3}q_{n-1} \\ q_0 = 1. \end{cases}$$

- a Implement the two relations in order to generate the first 100 terms of the sequence.
- b Study the stability of the two algorithms and justify the obtained results.

Homework 1.2

Find the minimum positive number representable in MATLAB by implementing an ad hoc procedure. Compare with **realmin**.

Homework 1.3

- a Use Taylor polynomial approximation to avoid the loss of significance errors in the following function when x approaches 0

$$f(x) = \frac{1 - \cos(x)}{x^2}$$

- b Reformulate the following function $g(x)$ to avoid the loss of significance error in its evaluation for increasing values of x towards $+\infty$

$$g(x) = x(\sqrt{x+1} - \sqrt{x}).$$

Homework (II)

Homework 1.4

We can compute e^{-x} around $x = 0$ using Taylor polynomials in two ways, either using

$$e^{-x} \approx 1 - x + \frac{1}{2}x^2 - \frac{1}{6}x^3 + \dots$$

or using

$$e^{-x} = \frac{1}{e^x} \approx \frac{1}{1 + x + \frac{1}{2}x^2 + \frac{1}{6}x^3 + \dots}.$$

Which approach is the most accurate?

Homework (III)

Homework 1.5

Consider the following integral

$$I_n(\alpha) = \int_0^1 \frac{x^n}{x + \alpha} dx, \quad \forall n \in \mathbb{N}, \alpha > 0.$$

- a Give an upper bound for $I_n(\alpha)$, $\forall n \in \mathbb{N}, \alpha > 0$.
- b Prove the following recursive relation between $I_n(\alpha)$ and $I_{n-1}(\alpha)$:

$$\begin{cases} I_n(\alpha) = -\alpha I_{n-1}(\alpha) + \frac{1}{n} \\ I_0(\alpha) = \ln\left(\frac{\alpha+1}{\alpha}\right) \end{cases}$$

- c Employing the previous relation, compute $I_{40}(\alpha = 8)$ and comment the obtained results.
- d Write a numerically stable recursive relation for $I_{40}(\alpha = 8)$.

Homework 1.6

Given the following sequence:

$$\begin{cases} x_{n+1} = 2^{n+1} \left[\sqrt{1 + \frac{x_n}{2^n}} - 1 \right] \\ x_0 > -1 \end{cases}$$

for which $\lim_{n \rightarrow +\infty} x_n = \ln(1 + x_0)$.

- a Set $x_0 = 1$, compute $x_1, x_2, \dots, x_{71}$ and explain the obtained results.
- b Transform the sequence in an equivalent one that converges to the theoretical limit.

References I